Image Classification Extension

https://github.com/aiyshwary/Image-Classification-Extension-in-Rapid-Miner

Python PyTorch ONNX RapidMiner Transfer Learning

OVERVIEW

A RapidMiner Studio extension enabling no-code transfer learning for image classification, supporting fine-tuning of popular CNN architectures on custom datasets and ONNX-based inference without writing a single line of code.

IMPACT

Developed a RapidMiner extension with two custom operators — Fine Tuning and Inference — enabling practitioners to apply transfer learning with ResNet, VGGNet, DenseNet, Inception and more using a drag-and-drop interface.

TECHNICAL WALKTHROUGH

1. Image Classification Extension

Problem Statement
In real-world scenarios, image datasets vary significantly in size, quality, and domain. A fixed model often fails to generalize well. Hence, there is a need for a customizable image classification system that can adapt to different datasets while maintaining good accuracy.

Approach
Built an image classification extension using TensorFlow and PyTorch Leveraged transfer learning with pre-trained models such as:

ResNet

AlexNet

VGG

SqueezeNet

DenseNet

Inception
Vision Transformers (ViT, DeiT via Hugging Face)

Fine-Tuning

Dataset organized in class-wise folders
Automatic 75% training / 25% validation split

Customizable parameters:

Number of layers

Batch size

Learning rate

Optimizer

Epochs

Used data augmentation, class-weighted learning, and callbacks
like early stopping and learning-rate scheduling

Inference

Trained model exported to ONNX
Images are preprocessed and passed to the model

Output includes:

Predicted class label

Confidence score
Images are automatically placed into output folders based on predicted class

Results
Animal classification: ~87% accuracy Facial emotion recognition: ~80% accuracy Flower dataset: ~94% accuracy

KEY DETAILS

- Fine Tuning operator accepts an image directory and hyperparameters; outputs a fine-tuned model in ONNX format.
- Inference operator loads the ONNX model and outputs a prediction DataFrame with classified labels.
- Supports ResNet, AlexNet, VGGNet, SqueezeNet, DenseNet, and Inception architectures.
- Compatible with both CPU and GPU (CUDA) environments via configurable conda environment.
- Installed as a JAR extension in RapidMiner Studio with Python Scripting backend.